

Documentation Développeur

Keep Your Head On

Groupe Nice 12 : Toby FOSTER, Lisa DIMITRIEF-DONTCHEFF, Chiara OLIVIERI

1. Introduction générale

1.1 Ce document a pour objectif de présenter de manière claire le fonctionnement interne de notre jeu « Keep Your Head On » du point de vue du développement.

Nous y expliquons l'architecture générale du projet, les technologies que nous avons choisies ainsi que la façon dont nos différents systèmes interagissent entre eux.

L'idée est de fournir une vision technique cohérente de l'ensemble du code : structure, flux de données, logique métier et organisation des principaux modules (combat, gestion de la monnaie, personnalisation, boutique, progression).

Nous nous concentrons ici uniquement sur les aspects techniques du projet.

Tout ce qui relève du game design pur, comme l'équilibrage des niveaux, la progression détaillée, les intentions artistiques ou la narration, est volontairement laissé de côté et pourra être détaillé dans un document séparé dédié à ces aspects.

1.2 Concept du jeu (vue développeur) : « Keep Your Head On » est un roguelike dans lequel le joueur affronte des vagues de plantes hostiles sur un terrain découpé en trois lignes (lanes). Les unités contrôlées par le joueur sont des créatures composées de parties de corps personnalisables (tête, bras gauche, bras droit, corps, jambes). Chaque partie de corps appartient à une classe parmi : Human, Zombie, Mythical, Robot, Daemon. Les grandes étapes d'une partie sont : - Sélection d'une classe de départ pour le joueur. - Lancement d'un niveau avec une arène à trois lignes. - Placement d'unités alliées en dépensant une monnaie interne. - Résolution automatique des combats sur chaque lane. - Si le joueur échoue : possibilité de recommencer le combat. - Si le joueur réussit : gain de monnaie. - Accès à la boutique (Shop) pour acheter de nouvelles parties de corps. - Accès à l'éditeur de personnage pour personnaliser ses unités. - Passage au niveau suivant, avec difficulté croissante. Une structure de progression de niveaux est déjà en place (via GameState et SpawEnemy) et permet d'introduire des ennemis plus difficiles ou des boss à partir de certains niveaux.

1.3 Boucle de jeu globale : D'un point de vue technique, la boucle de jeu suit la séquence suivante : - Lancement de l'application via Lwjgl3Launcher dans le module desktop (lwjgl3). - Initialisation de LibGDX et création de l'instance Main (classe principale). - Dans Main.create() : - Instanciation de GameState (état global de la partie). - Chargement des BodyPart via BodyPartLoader à partir du fichier JSON body_parts/body_parts.json. - Initialisation statique de Mob à l'aide du BodyPartLoader. - Création d'un SpriteBatch partagé pour le rendu 2D. - Mise en place de l'écran d'accueil (TitleScreen). À partir de là : - Main gère le changement d'écrans LibGDX (Screen) selon les actions du joueur et l'état du jeu. - Chaque écran est responsable de sa logique de rendu (render), de sa logique d'input et d'une partie de la logique métier. - GameState centralise la progression (niveau, argent, parties de corps débloquées, classe du joueur, etc.).

2. Technologies et architecture globale

2.1 Technologies principales Langage : Java - Le projet est intégralement écrit en Java. - Le typage fort et l'orientation objet facilitent le découpage du projet en classes et en packages. Framework de jeu : LibGDX LibGDX est utilisé pour : - La boucle de jeu (ApplicationListener / Game). - La gestion des écrans via l'interface Screen. - Le rendu 2D (SpriteBatch, Texture). - La gestion des polices (FreeTypeFontGenerator). - La gestion des entrées (clavier, souris, drag and drop). - La gestion de certains utilitaires (JsonReader, Preferences, etc.). Système de build : Gradle - Le projet utilise Gradle avec un wrapper (gradlew / gradlew.bat). - Les dépendances (LibGDX, LWJGL3, etc.) sont gérées dans build.gradle. - La structure de modules suit le modèle standard LibGDX : core + desktop (lwjgl3).

2.2 Modules du projet - Module core : - Package principal : io.github.abomination - Contient toute la logique de jeu multiplateforme : gameplay, écrans, état, gestion des lanes, etc. - Module lwjgl3 (desktop) : - Contient la classe de lancement Lwjgl3Launcher. - Configure la fenêtre (taille, titre) puis instancie Main. Cette séparation permet, à terme, de viser d'autres plateformes (Android, HTML5, etc.) sans modifier la logique du module core.

2.3 Classe principale : Main Package : io.github.abomination Rôle : - Main étend com.badlogic.gdx.Game. - Attributs principaux : - GameState gameState : état du jeu pour la session en cours. - SpriteBatch batch : batch global de rendu 2D partagé entre les écrans. - Dans create() : - Instancie un GameState. - Crée un BodyPartLoader à partir de Gdx.files.internal("body_parts/body_parts.json"). - Initialise Mob.initialize(loader) pour donner à la classe Mob l'accès aux définitions de parties de corps. - Instancie SpriteBatch. - Définit l'écran initial (new TitleScreen(this)). - Fournit la méthode getGameState() pour accéder à l'état global depuis tous les écrans. Justification des choix : - Étendre Game permet d'utiliser facilement setScreen() pour changer d'écran. - Centraliser SpriteBatch et GameState dans Main assure une cohérence des ressources partagées. - L'initialisation d'un BodyPartLoader unique évite le rechargement redondant du JSON de définition des parties de corps.

3. GameState : état global de la partie

3.1 Structure Classe : io.github.abomination.GameState Éléments principaux : - Enum interne GameScreen : GAME, SHOP, LEVEL_SELECTOR, CHARACTER_EDITOR. - currentScreen : écran logique courant (pas forcément un Screen LibGDX, mais un état métier). - level : entier représentant le niveau actuel du joueur. - money : monnaie virtuelle disponible. - playerClass : PlayerClass représentant la classe choisie par le joueur. - unlockedParts : collection de BodyPart disponibles pour la personnalisation. - Liste ou structures associées à la configuration des unités du joueur (composition des personnages). GameState propose notamment : - getCurrentScreen() / setCurrentScreen(GameScreen). - getLevel(), incrementLevel(). - getMoney(), addMoney(int), spendMoney(int). - getPlayerClass() / setPlayerClass(PlayerClass). - Des méthodes pour ajouter ou récupérer des BodyPart débloquées.

3.2 Rôle - Centraliser toutes les informations persistantes pendant une session de jeu. - Fournir une API claire pour les écrans : - ShopScreen utilise GameState pour connaître et modifier la monnaie et l'inventaire de parties de corps. - GameScreen lit la classe du joueur, le niveau actuel, et peut mettre à jour la monnaie selon les résultats des combats. - CharacterEditorScreen lit et modifie la composition des unités du joueur.

3.3 Intégration avec CurrencyManager GameState manipulera la monnaie de la session. Une classe séparée CurrencyManager se charge de la persistance de la monnaie globale via Gdx.Preferences : - getCoins() : lit le

nombre de coins depuis les préférences (clé "coins"). - addCoins(int) : ajoute la valeur et persiste. - spendCoins(int) : décrémente en s'assurant de ne pas descendre en dessous de zéro. Ce découplage permet d'avoir : - Une gestion de l'argent en mémoire pour la session (GameState). - Une persistance simple côté joueur (CurrencyManager).

4. Architecture des écrans (Screens LibGDX)

4.1 Principe général : Chaque écran visible par le joueur est une classe qui implémente com.badlogic.gdx.Screen : - TitleScreen : écran d'accueil. - ClassSelectScreen : choix de la classe principale (Human, Zombie, etc.). - LevelSelectScreen : gestion ou affichage du niveau courant. - GameScreen : écran principal de combat. - ShopScreen : boutique. - CharacterEditorScreen : personnalisation des personnages. - OptionsScreen : réglages divers. Chaque Screen suit le cycle de vie LibGDX : - show() : initialisation (création de Stage, chargement de polices, etc.). - render(float delta) : logique de mise à jour + appel au rendu graphique. - resize(), pause(), resume(), hide(), dispose().

4.2 Navigation entre écrans : La navigation passe toujours par Main : - game.setScreen(new TitleScreen(game)); - game.setScreen(new GameScreen(game)); - etc. Les triggers de changement d'écran sont : - Des actions utilisateur (clic sur un bouton, choix de classe, validation d'achat, bouton retour). - Des événements de gameplay (fin de combat, victoire d'un niveau, passage au shop, etc.).

4.3 Gestion de l'interface utilisateur : Plusieurs écrans utilisent : - La scène 2D (Stage) de LibGDX. - Des widgets Scene2D UI (Label, TextButton, Table, ScrollPane...). Par exemple, ShopScreen crée un Stage et un ScreenViewport, configure un BitmapFont, et fabrique un Skin minimaliste pour les labels et boutons. GameScreen utilise, en plus, des mécanismes de drag and drop (DragAndDrop) pour le placement d'unités sur les lanes. CharacterEditorScreen exploite une caméra OrthographicCamera, un FitViewport et un SpriteBatch pour afficher à la fois l'interface de sélection et les personnages. Une classe UI (io.github.abomination.UI) regroupe certaines responsabilités d'affichage et d'interaction pour éviter de surcharger les classes Screen avec trop de logique de dessin ou de layout.

5. Modèle de données : unités, lanes et combats

5.1 Classe Unit Package : io.github.abomination Unit est une classe générique représentant une entité pouvant exister sur une lane. Mob, EnemyMob et potentiellement d'autres entités en héritent. Les responsabilités typiques d'Unit sont : - Gérer une position (x, y) sur la lane. - Gérer une vitesse ou un déplacement. - Offrir des méthodes update() et draw() génériques.

5.2 Classe Mob : Mob étend Unit et représente une unité « vivante » composée de parties de corps : - Attributs principaux : - playerClass : PlayerClass (Human, Zombie, Mythical, Robot, Daemon). - head, leftArm, rightArm, body, legs : références BodyPart. - actualHealth : points de vie courants. - laneHeight : position verticale de la lane. - position horizontale, vitesse, direction, etc. - Constantes : - WIDTH, HEIGHT : dimensions logiques pour le rendu. Mob est également lié à BodyPartLoader via un champ statique bodyPartLoader initialisé dans Mob.initialize(loader). Il propose notamment : - update(float deltaTime) : mise à jour de la position, du cooldown d'attaque, etc. - getTotalHealth(), getTotalDamage(), getTotalSpeed() : calcul des statistiques finales en combinant les stats des BodyPart. - takeDamage(float amount) : réduit la santé

actuelle. - `isDead()` ou équivalent : indique si le mob doit être retiré de la lane. - `getPreviewTexture()` : renvoie une texture de prévisualisation.

5.3 EnemyMob et unités alliées EnemyMob : représente une unité contrôlée par le jeu (ennemi) et hérite de Mob. Les unités alliées peuvent être représentées soit par Mob directement configuré différemment, soit par des dérivés spécifiques. Les EnemyMob sont généralement instanciés par `SpawEnemy` en fonction du niveau actuel, puis ajoutés à une lane.

5.4 PlayerClass Enum : `io.github.abomination.PlayerClass` Valeurs : - Human - Zombie - Mythical - Robot - Daemon Chaque PlayerClass possède une liste des classes contre lesquelles elle est forte (`strongAgainst`). Exemples (d'après le code) : - Human est fort contre Zombie et Robot. - Zombie est fort contre Robot et Daemon. - Mythical est fort contre Human et Daemon. - Robot est fort contre Human et Mythical. - Daemon est fort contre Zombie et Mythical. PlayerClass fournit : - `isStrongAgainst(PlayerClass other)` : renvoie true si la classe courante est forte contre la classe passée en paramètre. - `getRandomStrongAgainst()` : renvoie une classe contre laquelle la classe courante est forte, de manière aléatoire. Cette logique est directement utilisée dans le système de combat pour moduler les dégâts.

5.5 Lane et LaneManager : Lane représente une ligne de combat : - Attributs principaux : - List units : unités présentes sur la lane (alliées et ennemies). - List activeCombats : combats actuellement en cours. - baseHealth : points de vie de la base du joueur sur cette lane. - baseDestroyed : booléen indiquant si la base est tombée. - height (hight dans le code) : position verticale de cette lane pour le rendu. Dans `update()` : - Chaque Unit est mise à jour (`update`) : - Si l'unité est un Mob, sa santé est vérifiée, et le mob est retiré si elle tombe à zéro. - Les collisions logiques entre unités de camps opposés peuvent déclencher la création de Combat. - Les combats actifs sont ensuite mis à jour via `updateCombats()`. - Si un ennemi atteint la base, baseHealth est réduite, et baseDestroyed peut devenir true. LaneManager : - Contient une List lanes. - Crée trois lanes dans son constructeur (pour les trois lignes du terrain). - Dispose d'une méthode `updateLanes()` qui appelle `update()` sur chacune des lanes. GameScreen possède un LaneManager et l'utilise pour mettre à jour l'ensemble des lanes et déterminer la fin du niveau.

5.6 Classe Combat : Combat modélise un affrontement entre deux Mob (`originalAttacker` et `originalDefender`). Principales responsabilités : - Calculer les multiplicateurs de dégâts en fonction des classes : - `attackerMultiplier` = 2.0 si la classe de l'attaquant est forte contre celle du défenseur, 1.0 sinon. - `defenderMultiplier` calculé de la même manière. - Gérer des cooldowns d'attaque pour chaque mob. - Mettre à jour l'état du combat à chaque frame via `update(float deltaTime)` : - Les intervalles d'attaque (`attackerInterval`, `defenderInterval`) sont calculés à partir des vitesses des mobs (`getTotalSpeed()`). - Lorsque le cooldown d'un mob dépasse son intervalle, il inflige des dégâts à l'autre (`health -= damage * multiplier`). - Le combat s'achève dès que l'un des deux mobs est mort. Cette séparation permet : - De tester et modifier la logique de combat indépendamment du reste du code. - D'ajouter ultérieurement d'autres mécaniques (critiques, dégâts sur la durée, effets spéciaux).

6. Système de personnalisation des personnages

6.1 BodyPart et BodyPartType Package : `io.github.abomination.body` BodyPart représente une partie de corps individuelle. Attributs typiques : - `id` : identifiant interne. - `name` : nom lisible du morceau. - `type` : BodyPartType (HEAD, LEFT_ARM, RIGHT_ARM, BODY, LEGS...). - `playerClass` : PlayerClass associée. - `health`, `damage`, `speed` : statistiques qui seront combinées pour calculer celles du Mob final. - Référence à une ressource graphique pour le rendu (Texture). BodyPartType est une enum qui catégorise les parties : - HEAD,

LEFT_ARM, RIGHT_ARM, BODY, LEGS, etc. Elle permet au code : - De filtrer les parties disponibles pour un slot donné. - De construire des personnages cohérents (une seule tête, deux bras, etc.).

6.2 BodyPartLoader : définition des données en JSON BodyPartLoader charge les définitions de parties de corps depuis un fichier JSON : - Utilise LibGDX JsonReader pour parser body_parts/body_parts.json. - Pour chaque entrée du fichier : - Lit id, name, type, playerClass, health, damage, speed. - Crée une instance de BodyPart correspondante. - Stocke ces BodyPart dans une structure accessible à Mob et aux écrans. Avantages : - Ajout de nouvelles pièces de corps sans toucher au Java : il suffit d'éditer le JSON. - Lisibilité des données pour les enseignants. - Possibilité d'équilibrer rapidement le jeu en jouant sur les valeurs de stats dans le JSON.

6.3 Calcul des statistiques d'un personnage : Le calcul repose sur la composition des BodyPart équipées sur un Mob. En général : - La santé totale est liée à la somme des health des parties. - Les dégâts totaux dépendent de la somme des damage. - La vitesse totale dépend de la combinaison des speed. Le code de Mob contient les méthodes getTotalHealth(), getTotalDamage() et getTotalSpeed() qui implémentent la formule exacte.

7. Boutique (Shop) et inventaire

7.1 ShopScreen et ShopItem : ShopItem est une classe simple qui représente un élément vendable : - name : nom de l'objet ou de la partie de corps. - price : coût en monnaie. ShopScreen est l'écran dédié aux achats : - Utilise Stage et ScreenViewport pour l'interface utilisateur. - Crée des Labels et TextButtons pour lister les items. - Affiche le nombre de coins disponibles (via GameState et/ou CurrencyManager). - Sur clic d'achat : - Vérifie que le joueur a suffisamment de monnaie. - Si oui, décrémente la monnaie et ajoute la BodyPart correspondante à l'inventaire. - Si non, peut afficher un message d'erreur ou ignorer l'action. La génération de la liste d'objets proposés peut utiliser : - Une sélection aléatoire parmi les BodyPart chargées. - Une logique plus avancée selon le niveau.

7.2 Gestion de la monnaie Deux niveaux de gestion : - En mémoire (GameState.money). - Persistant (CurrencyManager via Preferences). CurrencyManager : - getCoins(), addCoins(int), spendCoins(int). - Utilise Gdx.app.getPreferences("playerData") avec la clé "coins". GameState : - addMoney(int) et spendMoney(int) modifient la valeur en mémoire. - Il est possible de synchroniser régulièrement GameState.money avec CurrencyManager pour conserver une progression globale entre les sessions.

7.3 Intégration avec l'éditeur de personnage : Une fois les achats effectués : - Les nouvelles BodyPart sont ajoutées à l'inventaire stocké dans GameState. - CharacterEditorScreen lit cet inventaire pour afficher les pièces disponibles. - Le joueur peut alors combiner ces pièces pour créer des unités plus puissantes ou plus spécialisées.

8. Éditeur de personnage (CharacterEditorScreen)

8.1 Objectif : CharacterEditorScreen permet au joueur de : - Visualiser les personnages disponibles (ou au moins leur composition en parties de corps). - Parcourir l'inventaire des BodyPart débloquées. - Appliquer une BodyPart à un slot (tête, bras gauche, bras droit, corps, jambes). - Réinitialiser un personnage (supprimer toutes les personnalisations).

8.2 Implémentation technique CharacterEditorScreen : - Implémente Screen. - Utilise une OrthographicCamera et un FitViewport pour gérer la résolution et les coordonnées logiques. - Utilise SpriteBatch pour le rendu des éléments graphiques et BitmapFont / FreeTypeFontGenerator pour le texte. - Charge éventuellement une police spécifique (par exemple Knewave-Regular.ttf) si elle est présente dans les assets. Le flux utilisateur typique : - Le joueur sélectionne une catégorie (par exemple HEAD). - L'écran affiche toutes les BodyPart de type HEAD disponibles dans l'inventaire. - Le joueur clique sur l'une d'elles, puis clique sur le slot de la tête du personnage. - L'objet BodyPart est alors associé à ce slot dans la structure de données représentant le personnage.

9. Écran de combat (GameScreen)

9.1 Rôle GameScreen : est l'écran principal où se déroulent les combats : - Affiche l'arène avec trois lanes. - Affiche l'interface de sélection des unités à invoquer. - Gère la monnaie du joueur pour l'invocation. - Met à jour les lanes via LaneManager. - Contrôle la condition de fin de niveau (victoire ou défaite).

9.2 Implémentation GameScreen : - Implémente Screen. - Utilise : - Main game, GameState gameState. - LaneManager laneManager. - SpawnEnemy enemySpawner. - Une caméra et éventuellement un Stage pour l'interface. - Des mécanismes de DragAndDrop pour faire glisser des unités depuis un panel de sélection vers une lane. Le flux de combat est typiquement : - Au début du niveau : - GameScreen initialise LaneManager (trois lanes) et SpawnEnemy avec le niveau courant. - Le joueur dispose d'un certain nombre de coins de départ. - À chaque frame (render) : - Input : gestion des clics, drag and drop pour placer des unités. - Logiciel : - laneManager.updateLanes() met à jour tous les combats. - enemySpawner.update(deltaTime) décide s'il faut faire apparaître de nouveaux ennemis en fonction du temps et du niveau. - Rendu : dessin des lanes, des unités, des barres de vie, etc. - Conditions de fin : - Si toutes les plantes ennemies sont détruites et que la base du joueur n'est pas tombée : victoire. - Si la base (sur une lane ou globalement) est détruite : défaite.

9.3 SpawnEnemy : génération des ennemis SpawnEnemy prend en entrée : - Une liste de Lane (lanes). - Un niveau (level). - Un GameState (gameState). - Une PlayerClass (playerClass du joueur). Il gère : - timeSinceLastSpawn : temps écoulé depuis le dernier spawn. - Un intervalle de spawn dépendant du niveau : - baseSpawnInterval = 5.0f secondes. - minSpawnInterval = 1.0f. - spawnInterval = max(minSpawnInterval, baseSpawnInterval - level * 0.2f). Dans update(float deltaTime) : - timeSinceLastSpawn est incrémenté. - Si timeSinceLastSpawn >= spawnInterval, spawnEnemy() est appelé puis timeSinceLastSpawn est remis à zéro. - spawnEnemy() choisit une lane et y ajoute un EnemyMob configuré en fonction du niveau et de la classe.

10. Gestion des entrées (inputs)

10.1 Types d'entrées utilisés Le projet utilise principalement : - La souris : - Clic sur les boutons d'interface (TitleScreen, OptionsScreen, ShopScreen, etc.). - Clic sur les éléments de l'inventaire et sur les slots de personnage (CharacterEditorScreen). - Drag and drop des unités sur les lanes (GameScreen). - Le clavier : - Certaines touches peuvent être utilisées pour des actions globales (retour, pause, etc.).

10.2 Inputs spécifiques par écran - TitleScreen : - Boutons pour lancer une partie, accéder aux options, etc. - ClassSelectScreen : - Clic sur une classe pour la sélectionner dans GameState. - LevelSelectScreen : - Sélection d'un niveau (ou simple affichage et bouton pour continuer). - GameScreen : - Drag and drop d'unités depuis un panel vers une lane. - Éventuels raccourcis pour annuler une invocation ou revenir au menu. -

ShopScreen : - Clic sur les boutons d'achat. - CharacterEditorScreen : - Clic pour sélectionner une BodyPart. - Clic sur un slot pour l'affecter. - Boutons pour sauvegarder ou réinitialiser la configuration.

11. Tutoriel développeur : travailler sur « Keep Your Head On »

11.1 Prérequis - JDK installé (version compatible avec LibGDX, typiquement Java 8+). - Un IDE compatible Gradle (IntelliJ IDEA recommandé, Eclipse ou VS Code possibles). - Git (si le projet est versionné).

11.2 Import du projet 1. Cloner le dépôt contenant le projet. 2. Dans l'IDE, choisir « Import Project from Gradle » et sélectionner le répertoire racine T-JAV-501-NCE_12. 3. Attendre la résolution des dépendances. 4. Vérifier que les modules core et lwjgl3 sont bien reconnus.

11.3 Lancement du jeu 1. Créer une configuration d'exécution dans l'IDE pour la classe de lancement du module lwjgl3 (par exemple io.github.abomination.lwjgl3.Lwjgl3Launcher). 2. Exécuter cette configuration. 3. Le jeu se lance dans une fenêtre desktop avec le titre configuré et l'écran TitleScreen en premier.

11.4 Ajouter une nouvelle partie de corps 1. Ouvrir body_parts/body_parts.json. 2. Ajouter un nouvel objet de définition : - id unique. - name lisible. - type = "HEAD", "LEFT_ARM", etc. - playerClass = "Human", "Zombie", "Mythical", "Robot" ou "Daemon". - health, damage, speed avec des valeurs cohérentes. 3. Ajouter la ressource correspondante dans les assets si nécessaire. 4. Relancer le jeu : BodyPartLoader chargera automatiquement la nouvelle partie, et elle pourra être utilisée par le shop ou l'éditeur.

11.5 Ajouter un nouveau type d'ennemi 1. Créer une nouvelle classe dérivant de EnemyMob si un comportement spécifique est voulu, ou utiliser EnemyMob avec des paramètres différents. 2. Définir ses statistiques de base et sa classe PlayerClass. 3. Adapter SpawEnemy pour qu'il puisse instancier ce nouvel ennemi pour certains niveaux. 4. Tester en conditions réelles dans GameScreen.

11.6 Ajouter un nouveau niveau ou un boss 1. Décider comment représenter un niveau (par exemple via un entier level dans GameState). 2. Dans SpawEnemy et/ou dans GameScreen, ajouter des conditions basées sur level pour : - Changer le type d'ennemis générés. - Modifier la fréquence de spawn. - Introduire un boss (un EnemyMob avec de fortes stats et peut-être un comportement différent). 3. Mettre à jour la logique de fin de niveau pour gérer des conditions spécifiques (ex : tuer le boss plutôt que nettoyer toutes les lanes).

12. Choix techniques et justification

12.1 Java + LibGDX - Java est le langage étudié dans le cadre du cours et connu des enseignants. - LibGDX fournit un framework de jeu complet : - Gestion de la boucle de jeu. - Rendu 2D optimisé. - Gestion multiplateforme. - Outils utilitaires variés (polices, JSON, Preferences). Ce choix permet de concentrer les efforts sur la logique de jeu et la structure du code plutôt que sur la gestion bas niveau de la fenêtre et du rendu.

12.2 Séparation core / desktop (lwjgl3) - Le module core contient la logique générique et peut être réutilisé sur d'autres plateformes. - Le module desktop (lwjgl3) ne contient que la configuration spécifique à la plateforme (taille de fenêtre, titre, icônes, etc.).

12.3 Données externes (JSON) pour les BodyPart - Facilite l'édition des données sans recompilation. - Permet d'ajouter du contenu rapidement. - Rend la structure des personnages transparente pour un correcteur ou un game designer.

12.4 Architecture orientée composition de personnages - Représenter les personnages comme une composition de BodyPart rend le système très extensible. - Les statistiques d'un personnage sont directement la conséquence de ses composants. - Il est facile d'équilibrer ou de modifier certaines classes en ajustant les stats d'une seule BodyPart ou d'un groupe de BodyPart.

12.5 Lanes et gestion par LaneManager - Organiser le combat en lanes simplifie la logique : - Chaque Lane s'occupe de ses propres unités et combats. - LaneManager se contente de les mettre à jour toutes. - Cette structure facilite aussi l'affichage et la compréhension du code. **12.6 Système de combat encapsulé dans Combat** - Centraliser la résolution du combat dans une classe séparée évite de dupliquer la logique dans les mobs ou les lanes. - Le calcul des multiplicateurs de dégâts basé sur PlayerClass est facilement modifiable. - La classe est naturellement testable si l'on souhaite ajouter des tests unitaires.

13. Points à compléter et améliorables

Ce document couvre l'essentiel de l'architecture et des mécaniques techniques de « Keep Your Head On ». Certaines zones peuvent être complétées ou améliorées en fonction des attentes des professeurs ou de l'évolution du projet : - Détails exacts des formules de calcul des statistiques finales (santé, dégâts, vitesse) et justification de ces choix. - Description complète des raccourcis clavier et de la configuration d'options dans OptionsScreen. - Stratégie de sauvegarde et chargement des données (progression, configuration de personnages). - Schéma UML global du projet (classes principales, relations).

14. Conclusion

« Keep Your Head On » s'appuie sur une architecture Java/LibGDX structurée autour de : - Un état global centralisé (GameState). - Une séparation claire des écrans (Screen) pour chaque fonctionnalité majeure. - Un modèle de personnage basé sur la composition de BodyPart. - Un système de combat encapsulé dans la classe Combat, reposant sur les relations de force/faiblesse entre PlayerClass. - Un système de Boutique et d'Éditeur de personnage qui exploitent pleinement la structure des données. Cette organisation rend le code lisible, modulaire et raisonnablement simple à faire évoluer. L'ajout de nouveaux types d'ennemis, de nouvelles parties de corps, de boss ou de mécaniques supplémentaires peut se faire de manière incrémentale, en conservant la structure existante.